上海理工大学光电信息与计算机工程学院

2026—2027(1)

《计算机项目综合开发创新实践》
项目报告

项目名称　AI集成的智能招聘系统

姓　　名　杨紫雯

学　　号　2335060507

指导教师　张 艳

成　　绩：

目录

一、项目背景.....5
1.1 选题背景.....5
1.2 项目目标与意义.....5
二、需求分析.....6
2.1 角色划分.....6
2.2 功能性需求.....6
2.3 非功能性需求.....8
2.4 主要 AI 功能集成.....8
2.7 用例图.....9
三、总体设计.....12
3.1 系统架构.....12
3.2 功能模块图.....12
3.3 技术栈.....13
3.4 核心业务流程.....13
四、详细设计.....17
4.1 用户与认证.....17
4.2 简历模块.....17
4.3 职位与公司.....17
4.4 投递模块.....18
4.5 消息中心.....18
4.6 AI 集成.....19
4.7 管理员.....20
4.8 基础设施.....20
五、数据库设计.....21
5.1 数据库选型.....21
5.2 ER 图.....21
5.3 类图.....22
5.4 表清单.....22
六、系统实现.....23
6.1 开发环境与工具链.....23
6.2 关键技术实现.....23
6.3 关键模块或子系统运行流程描述.....25
6.4 部署与运行.....27
6.5 关键运行截图.....29
七、系统测试.....36
7.1 测试策略.....36
7.2 测试环境.....36
7.3 单元测试覆盖.....36
7.4 关键测试用例.....37
7.5 测试结果.....37
八、AI 工具使用情况说明.....38
九、总结.....38

9.1 项目成果.....38
9.2 技术亮点.....38
9.3 不足与展望.....39

项目题目：AI 集成的智能招聘系统

本系统是一个面向求职者、HR、管理员三类用户的智能招聘平台原型，基于 Spring Boot 3.5 + MyBatis-Plus 3.5 + Vue 3 + Element Plus 实现，集成大语言模型能力覆盖简历解析、简历↔职位评分、JD 润色、智能客服四大核心 AI 场景。系统共设计实现 8 大功能模块、20 张数据表、14 个 REST 控制器、19 个业务服务、25 个前端页面，配套 204 个后端单元测试全部通过。系统在完整业务流程（注册 → 简历 → 投递 →HR 推进 → 消息 → 面试）上完成端到端贯通，并在实际使用中针对发现的问题进行了多轮迭代修复。

一、项目背景

1.1 选题背景

互联网招聘行业经过多年发展已形成成熟的供需撮合模式，但在实际运行中仍存在若干结构性问题：(1) 信息过载——HR 发布的职位描述与求职者简历之间的语义匹配长期依赖关键词检索，难以捕捉深层语义关联；(2) 筛选成本高——HR 在初筛阶段需对每份简历投入 3-5 分钟阅读时间，单次招聘的简历处理量受人工瓶颈限制；(3) 匹配效率低——求职者主动投递的命中率较低，HR 与求职者之间的有效沟通往往发生在多次投递之后；(4) JD 撰写成本——HR 在发布职位时需撰写结构完整、关键词覆盖合理的 JD，本身就是一项专业工作；(5) 重复性咨询——求职者对招聘流程的高频问题（如何投递、进度查询、面试安排等）占用 HR 大量工作时间。

大语言模型在自然语言理解、结构化抽取、对话生成等任务上的能力，为上述问题提供了工程化解决方案。本项目正是面向这一场景，探索将 AI 能力集成到传统招聘 SaaS 中的完整实现路径。

1.2 项目目标与意义

本项目以"端到端可演示的智能招聘系统原型"为目标，覆盖以下范围：

在功能范围上，本项目涵盖账号体系、简历管理、职位发布、投递跟踪、消息中心、AI 集成、管理员审核 7 大核心模块。在角色覆盖上，系统服务求职者、HR、管理员三类典型用户，每类角色具备独立的注册/认证入口、功能权限与操作流程。在 AI 集成上，系统实现简历自动解析（AI-1）、简历-职位匹配评分（AI-2）、JD 一键润色（AI-3）、智能客服（AI-4）4 大场景的工程化调用链。

从工程意义看，本项目完整演示了一个具备 AI 能力的 SaaS 系统的端到端实现——从数据库设计、REST API 抽象、SSE 实时推送、前后端分离架构，到 AI 接口的接入与降级策略。从演示价值看，答辩时可现场演示完整业务流程，包含简历上传与解析、职位搜索与投递、HR 收件箱与状态推进、实时消息、AI 智能客服等核心场景。

二、需求分析

2.1 角色划分

系统按职能将用户划分为三类角色，对应不同的注册方式与权限范围。角色与注册方式对应关系如表 2-1 所示，其中管理员账号由系统预置（如 admin / system），其他角色支持公开注册。

角色	注册方式	核心定位
求职者	公开注册（邮箱+密码）	简历维护、职位搜索、投递、消息、AI 客服
HR	公开注册（含公司信息）	职位发布、收件箱、投递推进、消息
管理员	系统预置	用户管理、公司审核、字典维护

表 2-1

三角色在主要功能上的访问权限矩阵见表 2-2。整体设计原则是：求职者仅能操作自己创建的简历和投递；HR 可查看投递到本职位下的所有求职者简历，但不能干预求职者端的简历；管理员拥有全平台数据访问能力但侧重审核而非业务执行。

功能	求职者	HR	管理员
简历管理	自己的	查看（收件箱中）	查看（合规审核）
职位浏览	全部	全部	全部
职位发布	✗	自己的	✗
投递	自己投递	查看自己职位	全部
消息中心	与 HR 1:1	与求职者 1:1	✗
AI 客服	✓	✓	✗
用户管理	✗	✗	✓
公司审核	0✗	✗	✓
字典维护	✗	✗	✓

表 2-2

2.2 功能性需求

2.2.1 求职者端

求职者端覆盖 6 类功能：

(1) 账号：邮箱+密码注册（密码长度 ≥ 8 位，BCrypt 哈希存储）或个人资料维护，支持邮箱+验证码登录（开发态验证码打印到后端控制台）；

(2) 简历上传与修改：PDF/Word 上传（≤ 20MB），由 AI-1 解析为结构化 JSON（教育/工作/项目/技能/自我介绍），支持表单编辑、归档/删除/重新上传；

(3) 职位浏览：首页基于个人偏好+简历的 AI-2 推荐，结合列表筛选（关键词/行业/省份/城市/收藏），提供收藏、申请、详情、咨询 HR 等操作；

(4) 投递跟踪：按投递时间倒序的列表、状态时间线，撤回时向 HR 端发系统消息；

(5) 消息：与 HR 一对一聊天（投递后建立二元组唯一会话），支持附件上传/下载、SSE 实时推送、自动已读；

(6) AI 客服：全站悬浮按钮，关键词检索 + LLM 流式问答。

2.2.2 HR 端

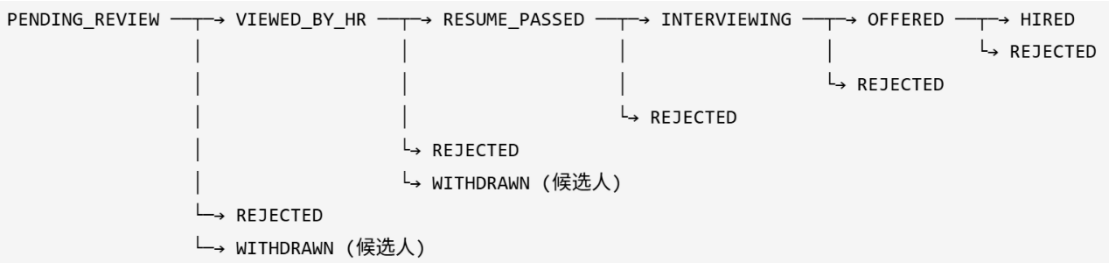
HR 端覆盖 4 类功能：
(1) 账号：邮箱+密码+公司信息（公司名必填，规模/简介可选）注册；
(2) 公司信息：行业/规模/简介自维护（修改已 VERIFIED 公司会自动重置为 PENDING 重新审核）；
(3) 职位管理：创建/编辑（草稿态）/发布（ONLINE）/下架（OFFLINE）/删除，AI-3 一键润色 JD（5 关键词 → LLM 生成）；
(4) 收件箱：按职位过滤投递列表、查看简历快照、状态推进（5 种合法目标）、HR 内部备注 CRUD，外加与投递过本职位（求职者可主动发起）的求职者 1:1 消息聊天。

2.2.3 管理员端

管理员端覆盖 3 类功能：
(1) 用户管理：列表（按角色/状态/邮箱筛选）、启用/禁用、重置密码（返回 6 位临时密码）、改角色（仅在求职者 ↔ HR 之间切换）；
(2) 公司审核：按状态筛选（默认 PENDING）、通过（→ VERIFIED）、驳回（必填理由，可选联动禁用对应 HR）；
(3) 字典维护：行业（一级 + 二级树形结构）、城市（含省份字段）、技能建议池的 CRUD。

2.2.4 投递状态机

投递记录用单一 application.status 字段承载 8 种状态，状态机如下：



终态为 HIRED / REJECTED / WITHDRAWN，不可再推进。求职者侧实际可看到所有非 HIRED 之外的状态，包括 VIEWED_BY_HR（以"已查看"展示），便于求职者了解 HR 处理进度；HR 侧可看到全部 8 种。撤回仅求职者可触发，终态不可撤回；撤回时向 HR 端发系统消息通知。

2.2.5 消息中心

消息中心按触发方与触发条件划分为 3 类场景：

触发方	触发条件	会话参与方
求职者	投递后或主动发起（半开放：求职者无需投递也可发起会话）	求职者-HR
HR	查看投递后或主动发起	HR-求职者
系统	求职者撤回投递	HR（系统消息）

系统在数据库层用 UNIQUE KEY uk_conversation_hr_candidate (hr_user_id,

candidate_id, is_deleted) 约束二元组唯一性——同一对“HR-求职者”永远只对应一个有效会话，无论求职者投过多少个职位。实时性方面，会话双方通过 SSE 长连接（/api/sse/subscribe）实时接收消息，服务端每 25s 发心跳注释防 nginx 60s proxy_read_timeout 切断连接。

2.3 非功能性需求

系统在性能、安全、可用性、可扩展性 4 个维度上的非功能性要求见表 2-3。

类别	指标
性能	列表查询 < 500ms；消息推送延迟 < 1s；AI 异步评分不阻塞 HTTP 返回
安全	JWT 鉴权（HMAC-SHA，jjwt 0.12.6）；密码 BCrypt 哈希；SQL 注入防护（MyBatis-Plus 参数化绑定）；密码 ≥ 8 位
可用性	角色权限矩阵、验证码登录（开发态降级到控制台日志）、SSE 断线自动重连
可扩展	AI 接口抽象为 LlmClient，切服务商仅改配置 URL/Key

表 2-3

2.4 主要 AI 功能集成

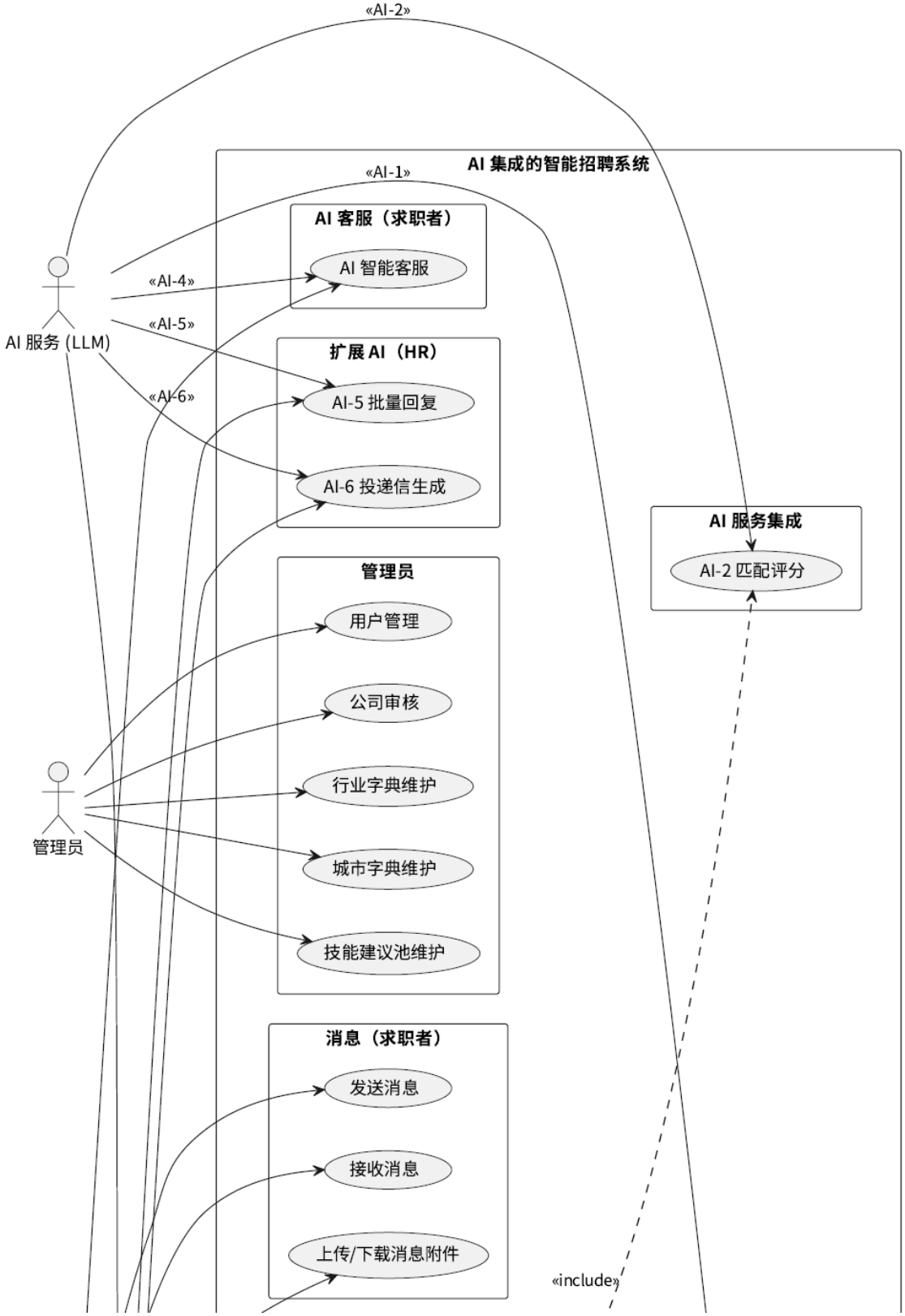
系统集成 4 个 AI 功能模块，分别承担不同的招聘环节智能化任务，详见表 2-4。

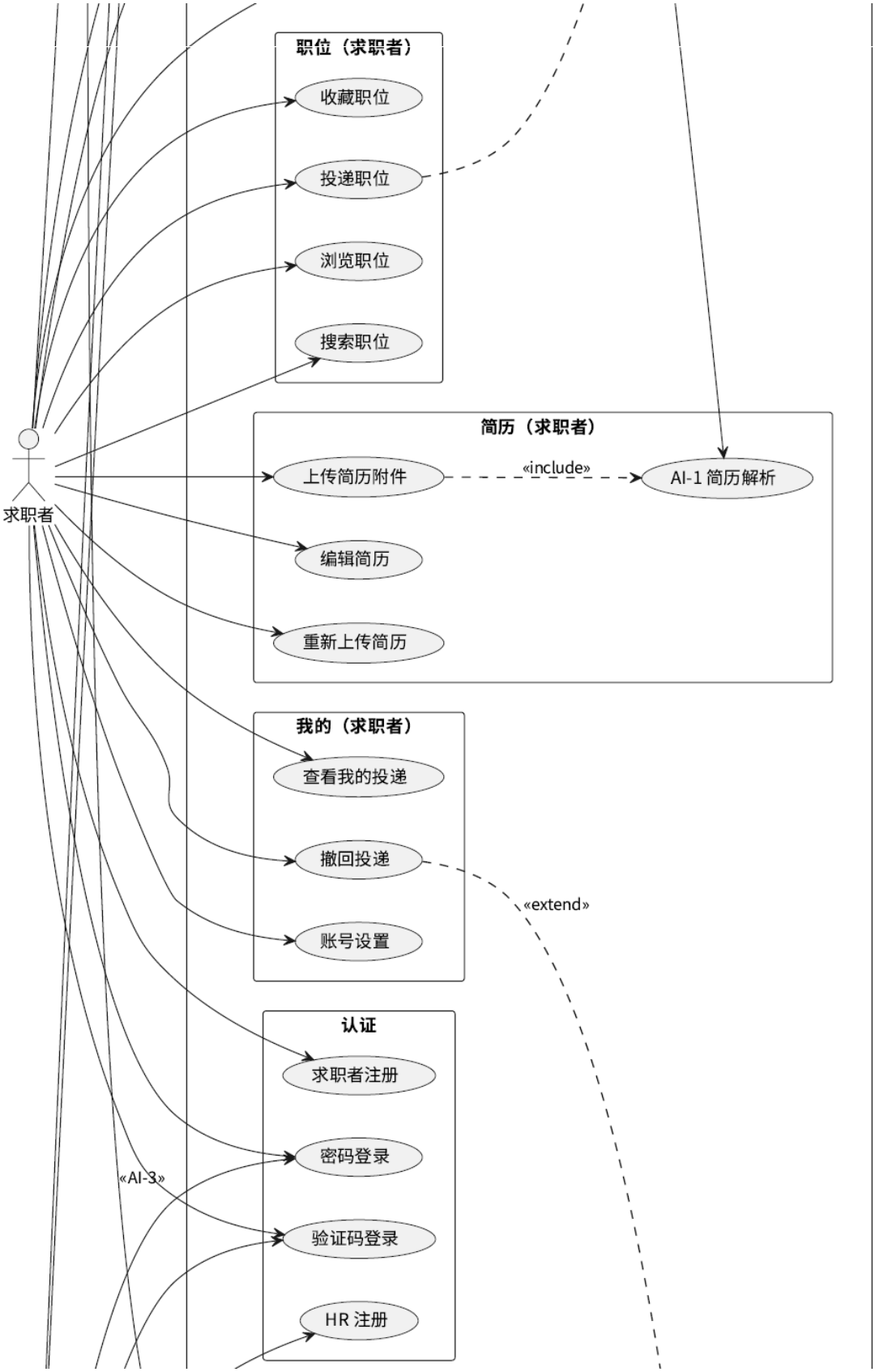
编号	名称	触发位置	同步 / 异步
AI-1	简历解析（将 PDF/Word 转换为结构化字段）	求职者上传附件瞬间（仅一次）	同步（前端等待 5-15s）
AI-2	简历-JD 匹配评分	求职者投递瞬间(仅一次) + 简历/偏好更新时	异步(不阻塞 HTTP 返回；结果回写 + 内存 cache + SSE 推送）
AI-3	JD 生成 / 润色	HR 编辑职位时主动触发	同步
AI-4	智能客服	全站悬浮聊天窗口	SSE 流式

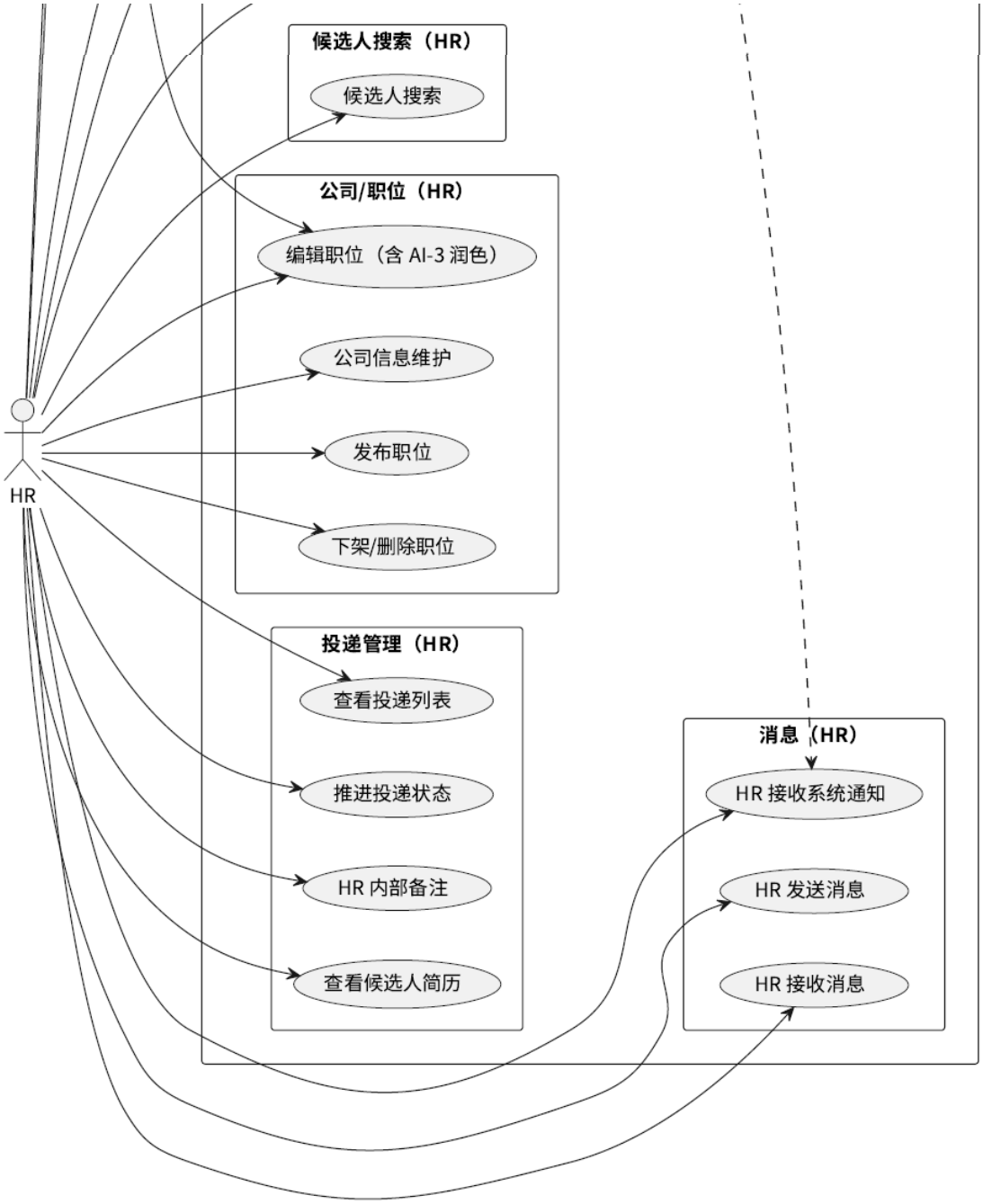
表 2-4

（AI-2 同时支撑两个使用场景：投递瞬间的单次评分（用于 HR 收件箱排序），以及求职者首页的"推荐职位"列表（基于偏好 + 简历）。两个场景复用同一套 LLM 调用与缓存。）

2.7 用例图

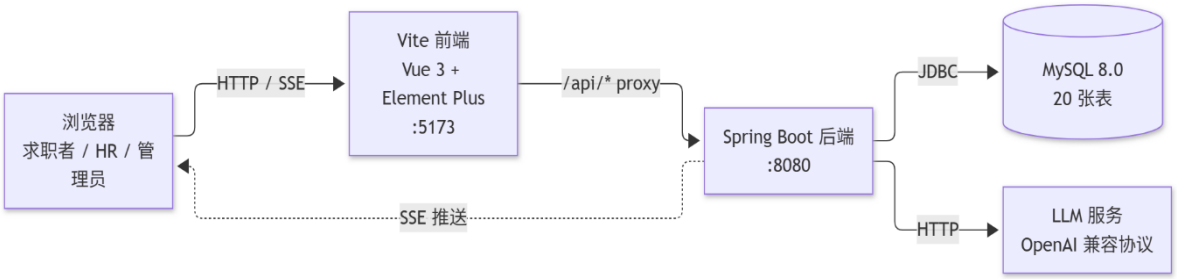

«AI-2»
AI 集成的智能招聘系统
«AI-1»
AI 客服（求职者）
AI 智能客服
«AI-4»
AI 服务 (LLM)
«AI-5»
扩展 AI（HR）
AI-5 批量回复
«AI-6»
AI-6 投递信生成
AI 服务集成
AI-2 匹配评分
管理员
用户管理
公司审核
管理员
行业字典维护
城市字典维护
技能建议池维护
消息（求职者）
发送消息
接收消息
上传/下载消息附件
«include»


职位（求职者）
收藏职位
投递职位
浏览职位
搜索职位
简历（求职者）
上传简历附件
«include»
AI-1 简历解析
编辑简历
重新上传简历
我的（求职者）
查看我的投递
撤回投递
«extend»
账号设置
认证
求职者注册
密码登录
«AI-3»
验证码登录
HR 注册
求职者


候选人搜索（HR）
候选人搜索
公司/职位（HR）
编辑职位（含 AI-3 润色）
公司信息维护
发布职位
下架/删除职位
投递管理（HR）
查看投递列表
推进投递状态
HR 内部备注
查看候选人简历
消息（HR）
HR 接收系统通知
HR 发送消息
HR 接收消息
HR

三、总体设计

3.1 系统架构

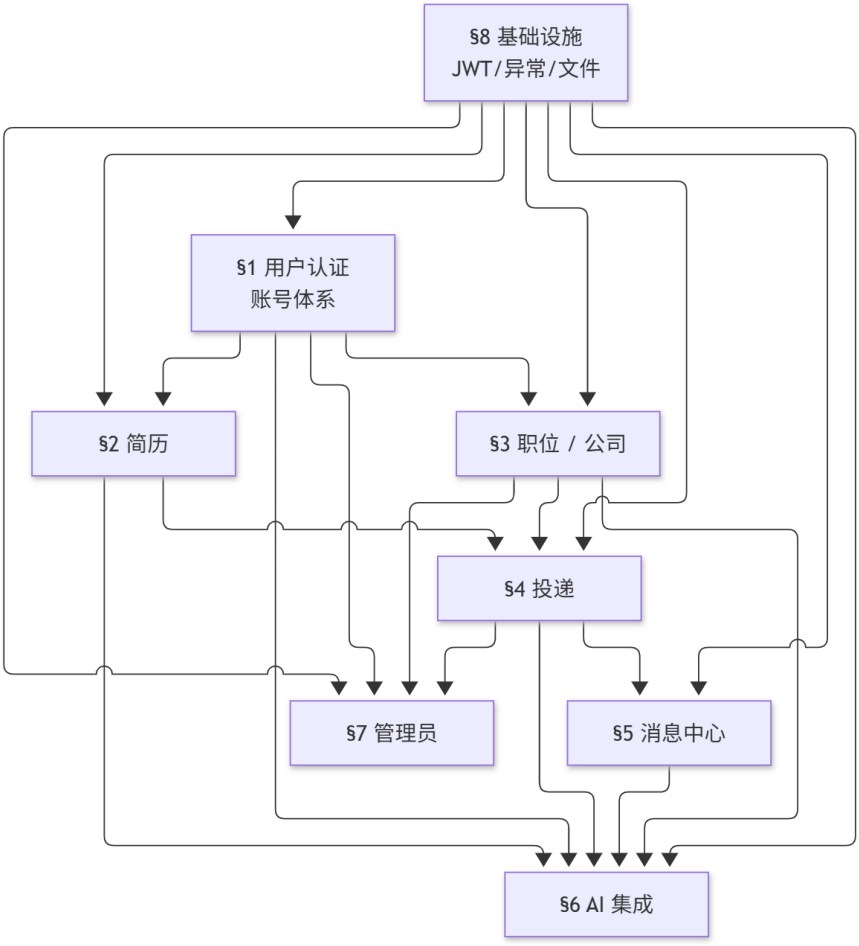


系统架构图

系统整体采用**前后端分离 + SSE 实时通道**架构，细节如下：

- 前端 Vue 3 单页应用，所有业务交互通过 /api/* REST 接口
- 后端 Spring Boot 统一响应包装 Result<T>{code, message, data}
- 实时性需求（消息推送、AI 异步评分结果）通过 SSE 推回前端
- AI 调用走标准 OpenAI 兼容协议，通过 LlmClient 抽象层统一调用

3.2 功能模块图



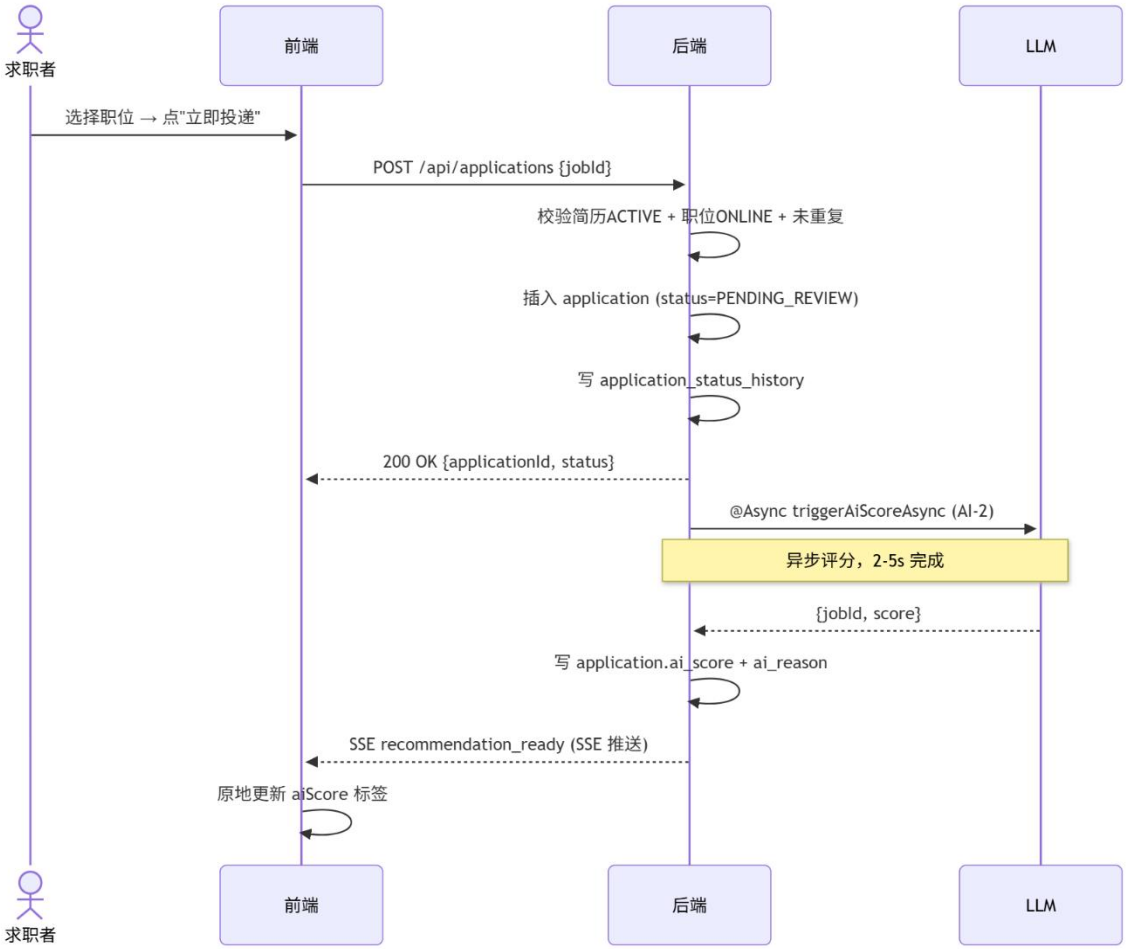
3.3 技术栈

层	技术	版本
后端框架	Spring Boot	3.5.14
持久层	MyBatis-Plus	3.5.7
数据库	MySQL	8.0.46
鉴权	jjwt	0.12.6
密码哈希	spring-security-crypto	跟随 Boot
简历文本抽取	Apache PDFBox + POI	—
前端框架	Vue	3.5.x
构建	Vite	8.x
状态	Pinia	4.x
路由	Vue Router	5.x
UI 组件	Element Plus	2.14.x
HTTP	Axios	1.x
AI	OpenAI 兼容协议	—

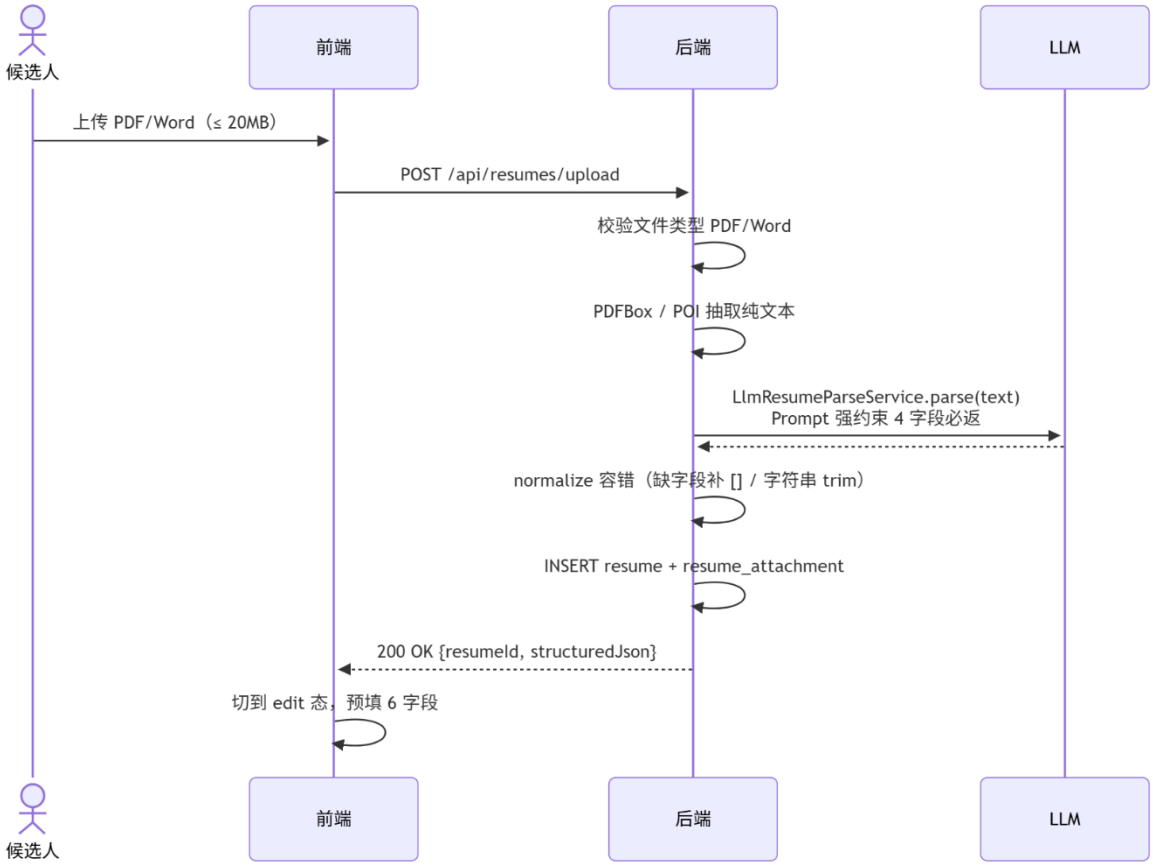
3.4 核心业务流程

本项目的核心业务流程时序图如下，主要聚焦于 AI 集成相关功能的实现：

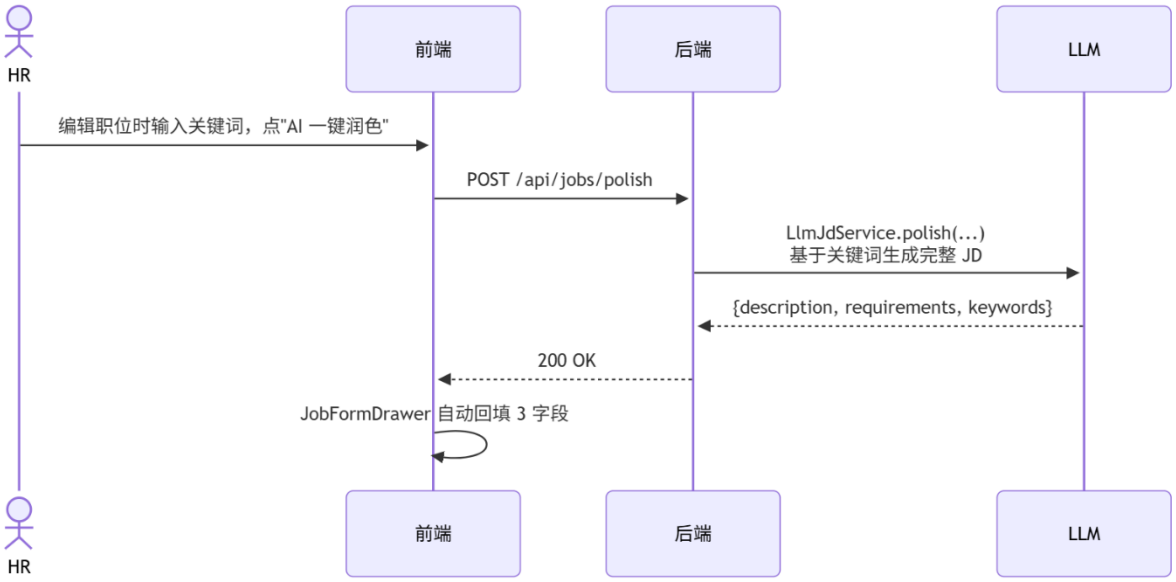
3.4.1 求职者投递流程（含 AI-2）



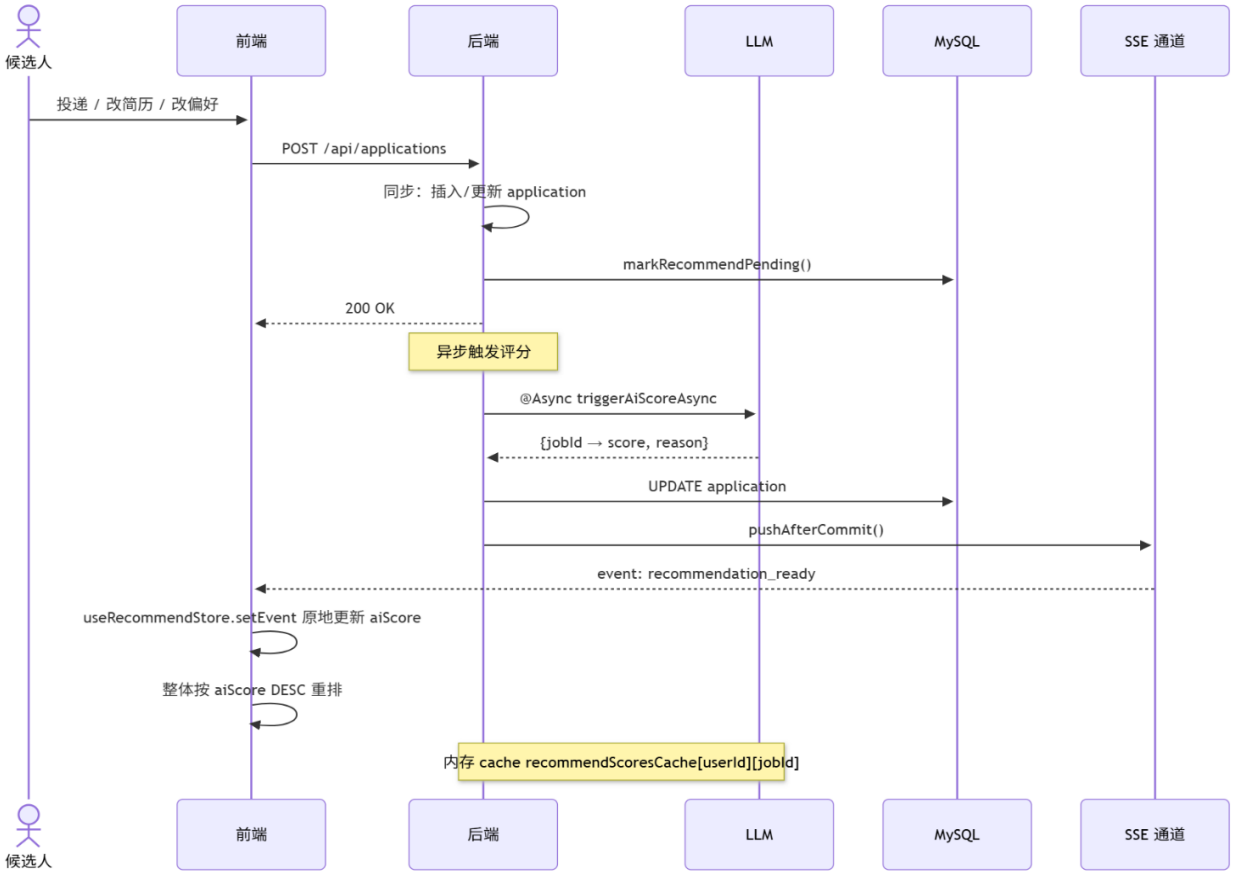
3.4.2 AI-1 简历解析流程



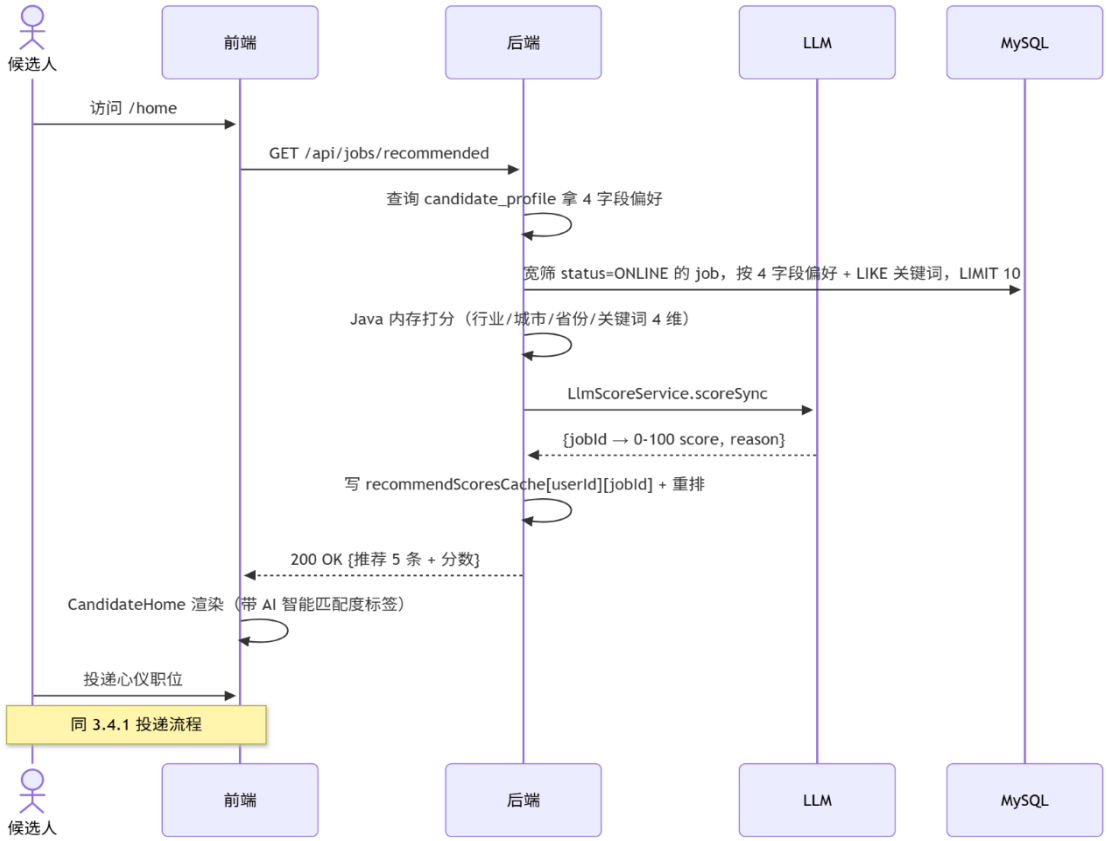
3.4.3 AI-3 JD 一键润色流程



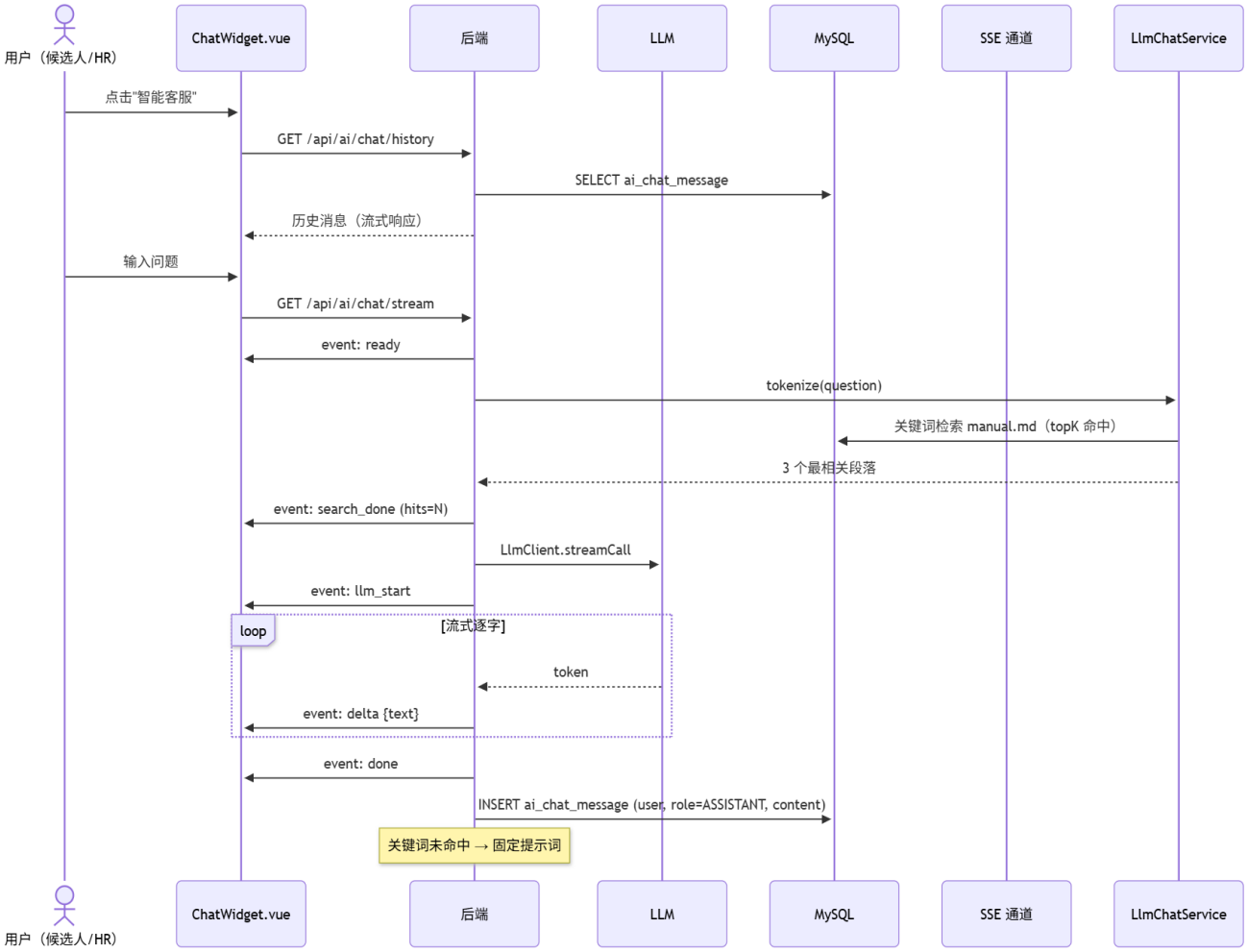
3.4.4 AI-2 简历与 JD 匹配评分流程



3.4.5 候选人首页推荐流程（含 AI-2）



3.4.6 AI-4 智能客服流程



四、详细设计

4.1 用户与认证

用户域核心实体为 users（账号主表）与 candidate_profile（1:1 扩展）。users 表关键字段包括 id（BIGINT PK）、email（UNIQUE 登录凭证）、password_hash（BCrypt 哈希）、role_code（CANDIDATE / HR / ADMIN）、status（ENABLED / DISABLED）、phone、username 及 created_at / updated_at。candidate_profile 为求职者偏好扩展，含 4 个偏好字段：expected_position（期望职位关键词）、expected_industry_id（FK → dict_industry）、expected_province、expected_city_id（FK → dict_city）。

鉴权流程采用 JWT + 角色注解模式：JWT 携带 sub(userId) + role 声明，密钥 ≥ 32 字节（HMAC-SHA 384，默认 24h 过期）；请求经 JwtFilter（解析 token）→ 注解校验（@RoleCandidate / @RoleHR / @RoleAdmin / @LoginRequired）两层过滤。客户端通过 Authorization: Bearer <token> 头携带 token；SSE 端点因浏览器原生 EventSource 不支持自定义头，改用 ?t=<token> 查询参数传递。

系统提供密码登录与验证码登录两种模式：密码登录（POST /api/auth/login）校验 BCrypt 哈希；验证码登录走「POST /api/auth/send-login-code → POST /api/auth/code-login」两步，验证码存 email_code 表（10 分钟有效），开发态 MailService 走控制台日志而非真实发邮件。

4.2 简历模块

简历模块前端 frontend/src/views/candidate/ResumeEdit.vue 实现三态切换：upload（拖拽上传卡，支持 PDF/Word，≤ 20MB）、view（卡片式展示教育/工作/项目三大区块 + 技能 tag + 自我介绍，底部 [重新上传] [编辑] [删除]）、edit（表单可输入，底部 [取消] [保存]）。每位求职者仅维护一份 ACTIVE 简历，由 is_archived 字段标记历史版本；删除时硬删附件磁盘文件但软删 DB 行（is_deleted=1 + @TableLogic）。

AI-1 解析由 ResumeServiceImpl.uploadAndParse 完成，5 步流程：(1) 文件类型校验（仅 PDF/Word）；(2) 文本抽取：将 PDFBox（PDF）/ POI（Word）转换为纯文本；(3) 调用 LlmResumeParseService.parse，解析提示词对 education / work / projects / skills 4 个数组字段做强约束，缺失或非数组时 normalize 强制转为空数组，日期统一 "YYYY-MM" 格式；(4) 解析结果容错：元素缺关键字段补 null、字符串 trim；(5) 落库 resume 表 + 落 resume_attachment 附件记录。

4.3 职位与公司

职位由 job.status 字段表达 3 态：DRAFT（草稿，求职者端不可见）、ONLINE（招聘中，求职者端可见）、OFFLINE（已下架，求职者端不可见）。HR 端按 3 状态分 tab 管理；求职者端 list 查询仅返 ONLINE 状态（status='ONLINE' AND is_deleted=0）。

AI-3 JD 润色由 JobController.polish 端点 + LlmJdService.polish 内部实现：入参为 title + industryName + cityName + salaryRange + keywords，调用 LLM 按

description / requirements / keywords 5 维度生成完整 JD，自动回填 JobFormDrawer 表单。HR 编辑职位时主动点"AI 一键润色"按钮触发。

HR 公司修改的 PENDING 重置由 ProfileServiceImpl.updateMyCompany 实现：检测到「已 VERIFIED」公司被修改任意字段时，自动重置 auth_status='PENDING' 并清空 verified_at/verified_by/auth_note。该规则确保 HR 不能通过改名绕过已通过的审核。

4.4 投递模块

ApplicationServiceImpl 严格校验 isValidTransition(from, to) 防止非法状态跳转；终态（HIRED / REJECTED / WITHDRAWN）不可再推进。求职者 / HR 端 DTO 字段分离通过 toDtoForCandidate 与 toDtoForHr 两个方法实现：求职者 DTO 不填充 aiScore / aiReason（安全合规，求职者不知道"被评分"这件事），HR DTO 填充 aiScore / aiReason / candidateName；共享字段为 id / jobId / status / appliedAt / withdrawn / withdrawable / jobTitle / companyName。

简历快照机制通过 application.resume_snapshot_id FK → resume.id 实现：求职者后续编辑或归档简历不影响历史投递记录的快照，AI-2 评分时取快照的当前内容。撤回流程由 withdraw 推进状态 + 写状态历史 + 调 MessageService.sendSystemMessage 通知 HR 三步完成；sendSystemMessage 用 @Transactional(propagation = REQUIRES_NEW) 隔离事务，失败不影响撤回主流程（保证 application.status 已变 WITHDRAWN 后再尝试通知 HR）。

4.5 消息中心

消息中心核心约束为二元组唯一：
UNIQUE KEY uk_conversation_hr_candidate(hr_user_id, candidate_id, is_deleted) 在 DB 层约束；ConversationService.findOrCreate 优先查询再插入。软删（is_deleted=1）不参与唯一性检查，但业务上不重新激活旧会话——撤回投递不删会话；删除求职者则硬删（删求职者前 cascade conversation）。

附件采用两阶段策略以避免 FK 孤儿：
(1) POST /api/messages/attachment/upload 仅落盘 + 解析 mime，返回 AttachmentUploadRef{fileName, filePath, fileSize, mimeType}（不入库）；(2) POST /api/messages 提交消息时携带 attachments:[ref],MessageServiceImpl.sendMessage 内部 bindToMessage(messageId, refs) 写 message_attachment 表。先有 messageId 再绑附件，保证 FK 关系有效。

SSE 长连接由 SseController.subscribe 注册 SseEmitter（timeout=0L 永不过期）到 SseEmitterManager.emitters[userId]。SseEmitterManager 维护 25s 心跳（应用层 : ping 注释）防 nginx 60s proxy_read_timeout 切断连接；onError / onTimeout / onCompletion 回调自动 unregister，无需手动清理。

自动已读由 ConversationService.open 在进入会话时执行：调 markPeerMessagesRead(conversationId, currentUserId) 把对方消息标 read_flag=1，再

推 SSE read_receipt 给对端（携带 readerId + messageIds）。

4.6 AI 集成

LlmClient 抽象层提供两个核心方法：

```
public class LlmClient {
    JsonNode callJson(String functionCode, String systemPrompt, String userPrompt, Long
callerUserId);
    void streamCall(String functionCode, String systemPrompt, String userPrompt,
                    Long callerUserId, Integer maxTokens, String modelOverride,
                    Consumer<String> chunkHandler);
}
```

其中 callJson 为同步调用，解析 JSON 返回，用于 AI-1 / AI-2 / AI-3；streamCall 为流式调用，chunkHandler 处理每段，用于 AI-4。请求头 response_format.type 必填（text 或 json_object），缺失则上游 HTTP 400。

LlmResumeParseService.parse（AI-1）调用 prompt ，对 LLM 输出做 4 项强约束：(1) education / work / projects / skills 4 个数组字段必须返回（即使原文为空也返 []）；(2) Few-shot 示例——附一段样本简历 + 期望输出；(3) 日期格式：统一为"YYYY-MM"，无法确定时返 null；(4) normalize 容错：缺失字段补[]、元素缺关键字段补 null、字符串 trim。

LlmScoreService.scoreSync（AI-2）支撑两个使用场景：(1) 投递瞬间单次评分：JobServiceImpl.triggerAiScoreAsync 在 @Async("taskExecutor") 线程中调用，结果回写 application.ai_score 与 ai_reason；(2) 求职者首页推荐：JobServiceImpl.recommendOnResume 按求职者偏好（4 字段）从 job 表宽筛前 N 条（≤ 10）后调 LLM 重排，aiScore 缓存到内存 recommendScoresCache[userId][jobId]。AI-2 触发条件：求职者改简历 / 改偏好时调 markRecommendPending(userId=true)；首次访问缓存为空时也触发；完成后通过 SSE recommendation_ready 事件推 aiScore 增量更新。前端 CandidateHome.vue 监听 recommendStore 变化，原地更新 aiScore + 整体按 aiScore DESC 重排，不重触发 loadRecommended()（避免"加载中…"闪屏）。

LlmJdService.polish(AI-3)以 5 个关键词为输入，LLM 生成完整 JD 的 description / requirements / keywords 三个字段，HR 编辑职位时主动点"AI 一键润色"按钮触发。

AiChatService.streamChat（AI-4）走 5 阶段 SSE 事件流（端点 /api/ai/chat/stream）：

```
1. ready          → 连接建立，前端可显示"就绪"
2. search_done    → 关键词检索完成，返回命中数
3. llm_start      → LLM 调用开始
4. delta ×N       → 流式返回 LLM token
5. done           → 完整响应结束
```

所有 AI 调用在 BusinessException 或通用异常时静默降级：(1) AI-1 失败：上传成功但简历字段为空，前端提示"AI 解析失败，请手动填写"；(2) AI-2 失败：ai_score =

NULL，HR 端展示"待评分"；(3) AI-3 失败：HR 可继续手动填写；(4) AI-4 失败：SSE error 事件，前端气泡显示"[出错]..."。降级路径不阻断主流程。

4.7 管理员

管理员提供 3 类操作：(1) 用户管理：列表按角色/状态/邮箱关键词过滤分页；启用/禁用改 users.status；重置密码生成 6 位随机临时密码（SecureRandom），弹窗仅显示一次给管理员；改角色仅在 CANDIDATE 和 HR 之间切换，不可改 ADMIN。(2) 公司审核：默认筛选 auth_status = PENDING；通过时记录 verified_at + verified_by（当前管理员 user_id）并置 auth_status=VERIFIED；驳回必填 auth_note 并置 auth_status = REJECTED；勾选「同步禁用对应 HR 账号」时同时改 users.status=DISABLED。(3) 字典维护：支持行业（一级 + 二级树形结构，parent_id 指向一级，CRUD 父级校验一级不能选父级）、城市（含 province 字段便于省份-城市二级联动）、技能建议池（求职者简历技能输入的补全源，API 已就绪但前端未做自动补全 UI）的 CRUD。

4.8 基础设施

后端统一返回 Result<T> 包装，code=200 表示业务成功，其他值表示业务异常。定义如下：

```
public class Result<T> {
    int code;          // 200 = 业务成功；其他 = 业务异常码
    String message; // 成功/失败消息
    T data;             // 业务数据
}
```

前端 Axios 拦截器统一解包：body.code===200 时，return body.data；否则弹 ElMessage.error(body.message)。

GlobalExceptionHandler(@RestControllerAdvice) 按异常类型分级处理：(1) BusinessException: Result.fail(e.getCode(), e.getMessage())
（HTTP 200，前端按 body.code 判断）；
(2) 校验异常:（MethodArgumentNotValidException / ConstraintViolationException）：Result.fail(400, ...)；(3) 异常抛出 Exception ：Result.fail(500, "系统异常，请稍后重试") + 完整堆栈日志。AI-4 SSE 端点特殊处理：AiChatService.streamChat 顶层增加 try/catch (Throwable) ，确保 SSE 通道不因异常而中断。

软删机制统一：所有业务表加 is_deleted TINYINT(1) DEFAULT 0 + @TableLogic 注解，MyBatis-Plus 自动在 SQL 拼 WHERE is_deleted=0，无需手写。文件存储方面，简历/消息附件存储在本地，路径："./uploads/{type}/{yyyy-MM}/{uuid}.{ext}"，由 FileStorageService 封装读写；删除 DB 行时不自动清磁盘。

五、数据库设计

5.1 数据库选型

系统选用 MySQL 8.0.46，理由：开源、InnoDB 事务与外键完整；utf8mb4 / utf8mb4_0900_ai_ci 字符集支持完整 Unicode（含 emoji）；与 AI 简历解析的 JSON 输出天然契合（JSON 列类型）。时区统一 Asia/Shanghai。

5.2 ER 图

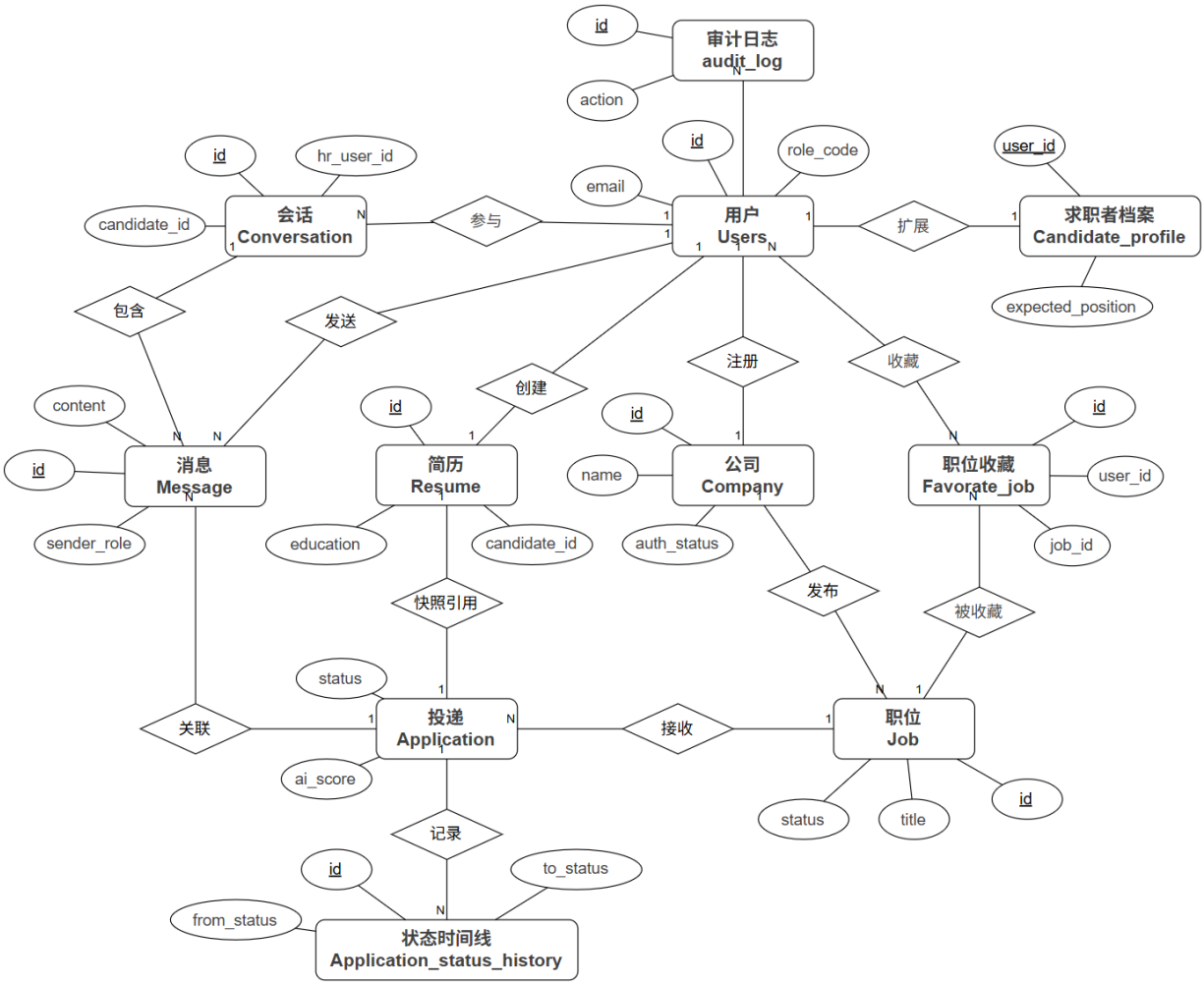


图 5-2 系统 ER 图（仅主要表）

5.3 类图

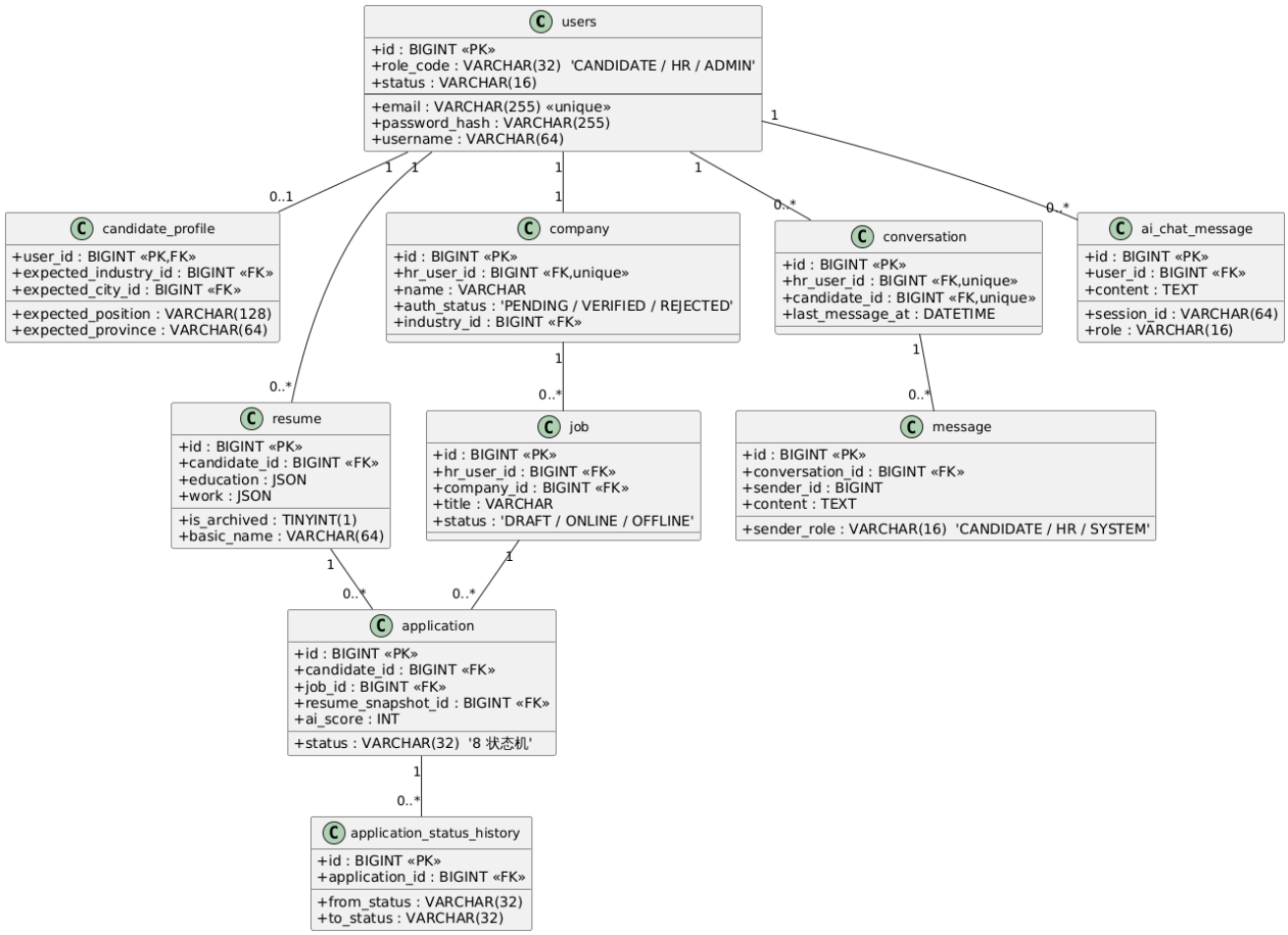


图 5-3 系统数据库类图（仅主要表）

5.4 表清单

完整数据库结构共 20 张表，按 10 个主题域划分如下：

域	表数	表名
用户	2	users, candidate_profile
简历	2	resume, resume_attachment
职位	2	company, job
投递	3	application, application_status_history, application_note
收藏	1	favorite_job
消息	3	conversation, message, message_attachment
字典	3	dict_industry, dict_city, dict_skill_suggestion
认证	1	email_code（邮件验证码）
AI	2	ai_call_log, ai_chat_message
审计	1	audit_log

六、系统实现

6.1 开发环境与工具链

类别	工具
JDK	17 LTS
构建	Maven 3.9
IDE	IntelliJ IDEA
数据库	MySQL 8.0
前端构建	Node.js 18+ / Vite 8
版本控制	Git + GitHub

6.2 关键技术实现

6.2.1 前端技术

前端采用 Vue 3.5 + Vite 构建工具链，技术构成如下：

- **Vue 3.5 + <script setup>**：所有页面与组件统一使用 Composition API 与 <script setup> 语法，组件逻辑内聚、模板与逻辑分离。
- **状态管理**（Pinia 4.x）：7 个 Pinia store 承担 user / resume / job / application / conversation / message / ai-chat 等领域状态，按模块拆分，避免单一 store 承载过多状态。
- **路由**（Vue Router 5）：基于角色的路由守卫在导航前校验身份与权限，未登录用户访问受保护页面会被重定向到登录页。
- **UI 组件库**（Element Plus 2.14）：覆盖表单、表格、分页、上传、消息提示等通用场景，视觉风格与企业级系统一致。
- **HTTP 客户端**（Axios）：拦截器统一解包 Result<T> 业务响应，401 触发自动跳转登录，错误码统一映射为 Element Plus 消息提示。
- **SSE 订阅**：浏览器原生 EventSource（自动重连），URL 通过查询参数“?t=<token>”传递 JWT（因 EventSource 不支持自定义 header），事件分发至各 Pinia store。
- **构建代理**：Vite 配置 proxy: { '/api': { target: 'http://localhost:8080' } }，开发期跨域由前端代理统一处理。

6.2.2 后端技术

后端基于 Spring Boot 3.5.14，技术构成如下：

- **Spring Boot 自动配置**：项目采用 application.yml + application-local.yml 双 profile 机制，敏感信息（数据库密码、LLM Key 等）集中放置于 application-local.yml（由 .gitignore 排除），避免敏感信息随仓库泄露。

- **ORM 与分页**（MyBatis-Plus 3.5.7）：通过 MybatisPlusConfig 注册 PaginationInnerInterceptor，所有 Service.selectPage 调用自动追加 LIMIT；CRUD 由 BaseMapper 提供，复杂条件优先采用 Lambda 表达式。统一使用 @TableLogic 注解实现软删过滤，业务表无需手工拼 WHERE 语句。
- **鉴权**（JWT 0.12.6）：所有受保护端点经 JwtAuthenticationFilter 校验，密钥 ≥ 32 字节（HMAC-SHA 384），令牌默认 24 小时过期；令牌中携带 userId 与 role，用于 Controller 层 @PreAuthorize 注解的细粒度权限控制。令牌生成核心代码如下：

```
String token = Jwts.builder()
        .setSubject(String.valueOf(userId))
        .claim("role", role)
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + 24 * 3600 * 1000))
        .signWith(SecretKey, Jwts.SIG.HS384)
        .compact();
```

- **实时通道（SSE）**：服务端 SseEmitter 由 SseEmitterManager 统一管理，配合 ScheduledExecutorService 每 25 秒发送心跳防 nginx 60 秒超时切断；事务提交后推送在钩子内触发，避免竞态。
- **AI 接口抽象**：所有 LLM 调用经 LlmClient 抽象层，提供 callJson,streamCall 两种接口,对外屏蔽 OpenAI 兼容协议的差异;切换服务商仅需修改 llm.api-url / llm.api-key/llm.model 三个字段，业务代码无须变更。核心类结构如下：

```
public class LlmClient {
        private final LlmConfig llmConfig;
        private final AiCallLogMapper aiCallLogMapper;
        private final RestClient restClient;
        private final ObjectMapper objectMapper;
        private final Cache<String, JsonNode> cache;
        private final HttpClient httpClient;

        public JsonNode callJson(String functionCode, String systemPrompt, String
userPrompt, Long callerUserId) {
        }
}
```

- **统一响应**（Result）：所有 Controller 返回 Result <T> 包装（code / msg / data 三段），拦截器在 HTTP 层做认证与日志，全局异常由 GlobalExceptionHandler 转为标准错误响应。

6.2.3 数据库实现技术

数据库层基于 MySQL 8.0.46 + InnoDB，技术构成如下：

- **存储引擎**：所有业务表统一使用 InnoDB，支持事务、行级锁与外键约束；字符集与排序规则在数据库级与表级均显式声明为 utf8mb4 / utf8mb4_0900_ai_ci。
- **ORM 集成**：通过 MyBatis-Plus 实体类与数据库表映射，复杂查询可使用 @Select 等注解配合参数化绑定，禁止字符串拼接；Page<T> 类型实现 pageNum / pageSize 风格的分页入参。
- **结构化数据存储**：简历的数组字段、AI 解析结果等结构化数据采用 MySQL JSON 列存储，并由应用层做字段级 normalize 容错，避免依赖数据库 JSON 函数。
- **索引与唯一键**：命名规范统一：唯一键 uk_<table>_<col>、普通索引 idx_<table>_<col>、外键 fk_<table>_<ref>;。
- **时间字段约定**：created_at / updated_at 统一，由数据库自动维护。
- **幂等迁移**：增量 DDL 脚本使用 CREATE TABLE IF NOT EXISTS 等幂等语句，可重复执行；初始化脚本一次性创建全部 20 张表。

6.3 关键模块或子系统运行流程描述

6.3.1 简历上传 + AI-1 解析子系统

子系统职责：将求职者上传的 PDF / Word 简历解析为结构化字段并落库，整个过程在用户上传的瞬间同步完成（5 - 15 秒）。

运行流程：

1. 求职者在前端上传页提交 PDF / Word 文件，前端先做本地校验（大小 ≤ 10MB、扩展名白名单），通过后调用 /api/resumes/upload。
2. Controller 层接收 MultipartFile，按文件扩展名白名单校验（仅允许 PDF / Word）；校验失败直接返回 400。
3. ResumeServiceImpl.uploadAndParse 调用 textExtractService.extract(file)，按扩展名分发到 PDFBox(PDF)或 POI(Word) 抽取纯文本；抽取失败抛 BusinessException。
4. 调用 LlmResumeParseService.parse(text) 触发 AI-1 简历解析：系统提示词强约束 education / work / projects / skills 四个数组字段必返回（即便原文为空也返 []），日期统一 "YYYY-MM" 格式。
5. 对 LLM 返回做 normalize 容错——缺失字段补 []、元素缺关键字段补 null、字符串 trim，避免非法内容入库。
6. 落库 resume 表（结构化字段）+ resume_attachment 表（附件路径与元数据）；返回 resumeId 与结构化 JSON 给前端。
7. 降级策略：AI-1 解析失败时，上传视为成功，简历字段为空，前端提示"AI 解析失败，请手动填写"；不阻断主流程。

6.3.2 投递 + AI-2 异步评分子系统

子系统职责：求职者发起投递请求后，先同步完成投递校验与落库，再异步调用 AI-2 完成简历 ↔ JD 评分，并把结果回写到投递行；HR 端可通过 SSE 实时收到评分结果。

运行流程（同步段）：

1. 求职者点击"投递"按钮,前端携带 jobId 与 resumeSnapshotId 调用/api/applications。
2. Controller 校验身份与请求体后进入 ApplicationServiceImpl.apply，方法标注 @Transactional 开启事务。
3. 校验三件事：① 当前职位状态必须为 ONLINE（非 ONLINE 抛"该职位当前不可投递"）；② 求职者必须有当前激活简历（否则抛"请先上传简历"）；③ 已投递过该职位则抛"已投递过该职位"（由 countActiveApplication 配合 idx_app_candidate_job 防并发）。
4. 构造 Application 实体，填充 candidate_id/job_id/resume_snapshot_id/status=PENDING_REVIEW/ai_score=NULL 等字段，写入 application 表。
5. 调用 recordHistory 向 application_status_history 表追加一条 null-PENDING_REVIEW 的记录，备注"求职者初次投递"，保留完整状态时间线。

运行流程（异步段）：

1. 同步方法返回前，调用 triggerAiScoreAsync(appId, candidateId, job, activeResume) 投递至 taskExecutor 线程池，立即返回 ApplyResponse 给前端。
2. 异步方法内调用 LlmScoreService.scoreSync(resumeJson, jobTitle, jobDescription, jobRequirements, candidateId)，由 LlmClient.callJson 完成 LLM 调用。
3. 评分成功后调用 onAiScoreCompleted 把 score 与 reason 回写到 application 表，并通过 recommendScoresCache 内存缓存 + SSE recommendation_ready 事件推送给求职者。
4. 降级策略：AI-2 评分失败时静默吞异常，application.ai_score = NULL 保留，HR 端展示"待评分"，下一次用户行为可触发重试；不阻断主流程。

6.3.3 消息中心 SSE 实时推送子系统

子系统职责：求职者和 HR 通过会话收发消息，新消息、撤回、已读回执、AI 评分结果等事件均通过 SSE 实时推送到对方客户端。

运行流程：

1. 用户在前端消息输入框提交消息，携带 conversationId 与内容（可选附件 ID 列表）调用 /api/messages。
2. MessageServiceImpl.sendMessage 校验调用方身份与会话归属（防止越权），调用 requireConversation 确认会话存在且当前用户是对端之一。
3. 构造 Message 实体并插入 message 表；若请求体带附件 ID，调用 attachmentService.bindToMessage 完成两阶段绑定（先上传再绑定，避免孤儿附件 FK）。

4. 更新 conversation.last_message_at = now，会话列表排序按此字段。
5. 计算对端用户 ID（HR 发送则推 candidate，反之推 HR）。
6. 调用 pushAfterCommit(peerId, "new_message", event) 注册一个事务同步器——只有当当前事务成功提交后，才向 SseEmitterManager 推 SSE 事件 new_message，事件载荷为 conversationId + 完整 MessageDto。该机制避免"接收方收到推送但读不到"的竞态。
7. 保活与断线重连：SseEmitterManager 每 25 秒向所有连接发送心跳注释行（:），防止 nginx 默认 60 秒空闲超时切断 SSE 通道；浏览器端原生 EventSource 在连接断开后自动重连，重连后由前端 store 重新订阅所需频道。

6.4 部署与运行

6.4.1 本地启动步骤

```
# 1. 启动 MySQL
net start MySQL80

# 2. 初始化数据库
mysql -uroot -p < docs/DataBase/schema.sql

# 3. 启动后端
cd backend
mvn spring-boot:run              # 监听 :8080

# 4. 启动前端
cd frontend
npm install
npm run dev                      # 监听 :5173
```

6.4.2 数据库初始化

项目使用全量 DDL 脚本一次性创建 20 张表。后续如需新增表或调整字段，使用配套的增量迁移脚本(可按需采用 CREATE TABLE IF NOT EXISTS 等幂等语句保证可重复执行)。

6.4.3 LLM API Key 配置

本项目使用的API Key 配置位于配置文件 application-local.yml，统一格式：

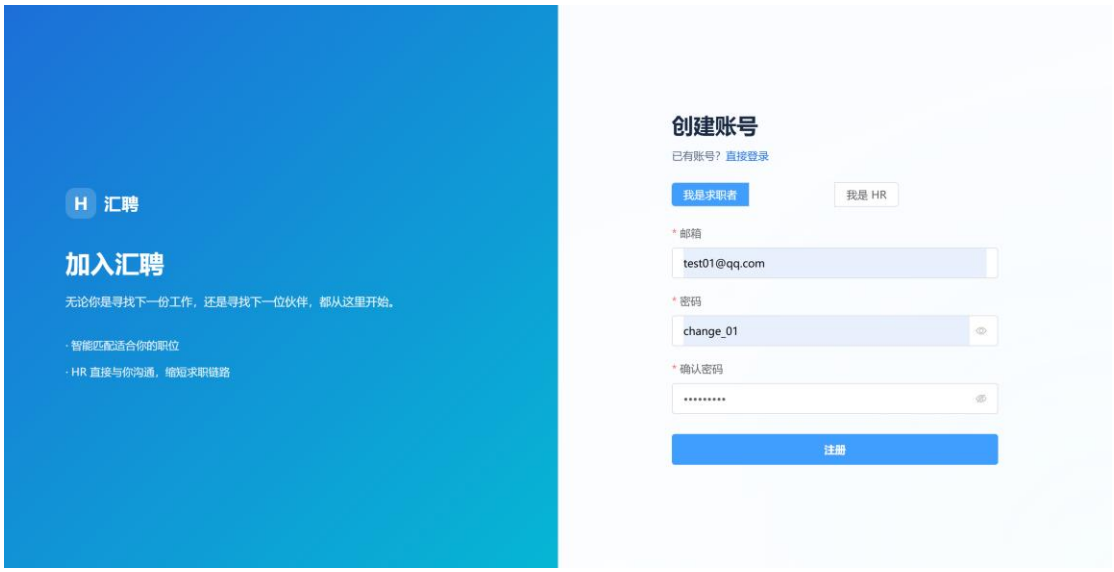
```
llm:
  api-url: ${LLM_API_URL: https://{your_url}/v1/chat/completions}
  api-key: ${LLM_API_KEY: {your_key}}
  model: {your_model}
  chat-model: ${LLM_CHAT_MODEL: {your_model}}   # AI-4 独立模型（速度优先）
  chat-max-tokens: 600
```

6.4.4 端口与代理

- 后端：8080
- 前端：5173（vite.config.js 中的 proxy: {'/api':{target:'http://localhost:8080'}}）
- SSE长连接通过 ?t=<token> 查询参数传token(因EventSource不支持自定义 header)

6.5 关键运行截图

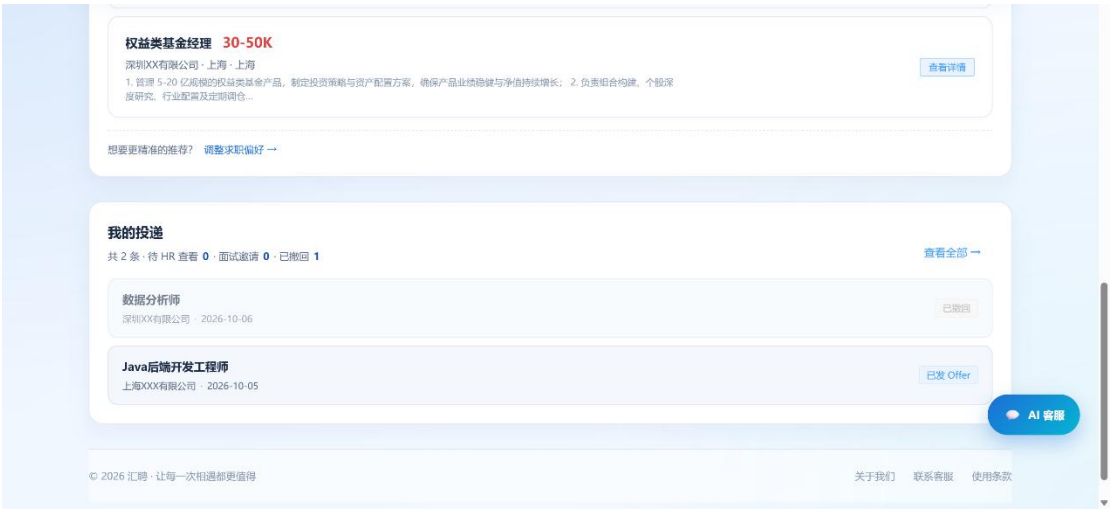
（1）注册页（求职者/ HR）



（2）求职者首页







（3）简历编辑页（未上传/已有简历 + 编辑态）









（4）职位浏览页（常态/搜索/只看收藏）





（5）职位详情页



（6）HR 职位管理页



（7）HR 职位发布

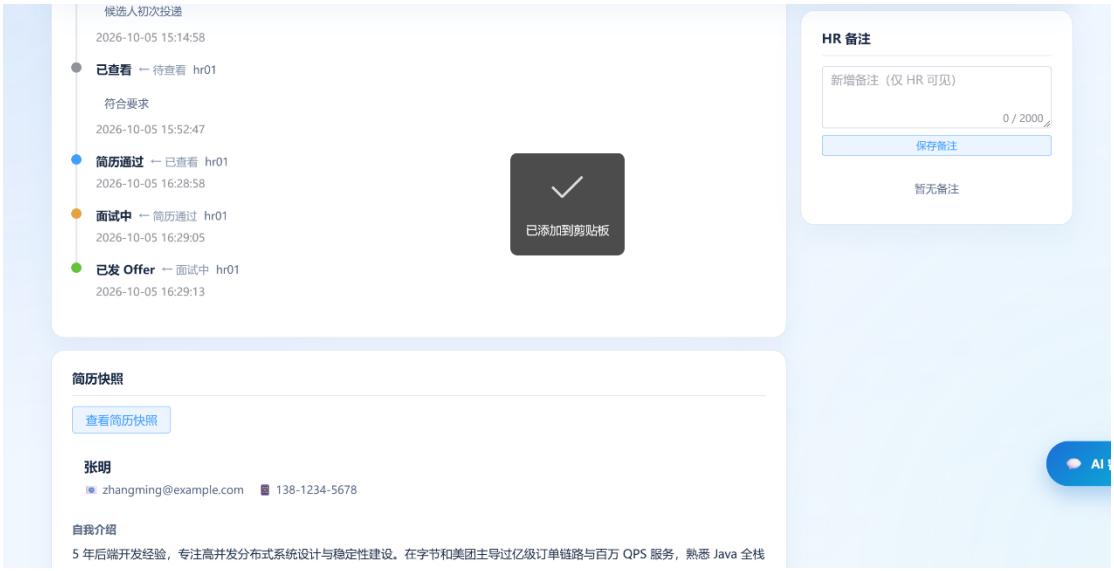


（8）HR 简历收件箱



（9）HR 投递详情页（投递推进时间线 + 求职者简历快照）





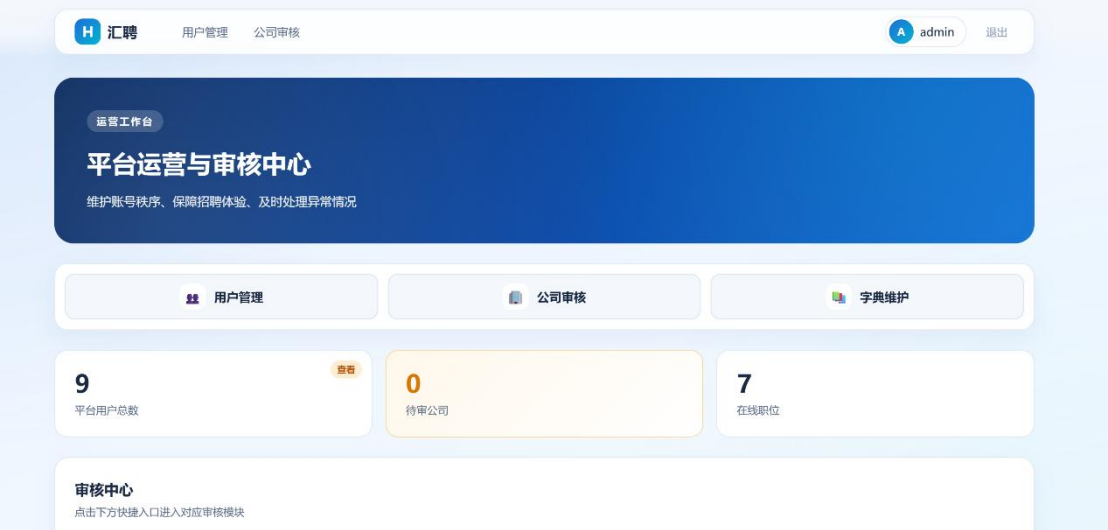
（10）消息中心页



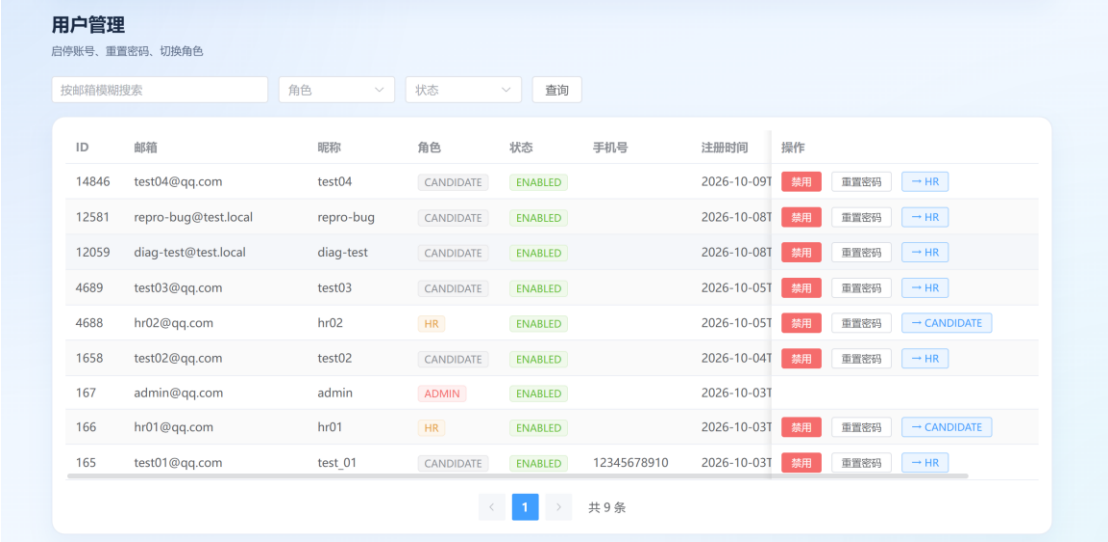
（11）AI 智能客服（弹窗模式）



（12）管理员首页



（13）管理员用户管理页



（14）管理员公司审核页



七、系统测试

7.1 测试策略

层级	方法	覆盖目标
单元测试	JUnit 5 + Mockito	业务 service / mapper / 工具类
集成测试	@SpringBootTest + @Transactional 回滚	完整 service 链路（含 mapper 真实 SQL）
端到端冒烟	浏览器 + curl	完整业务流程（求职者 + HR + 管理员三角色联动）

7.2 测试环境

- JDK 17 + MySQL 8.0 + Spring Boot 3.5
- 测试 DB：devplogin / devplogin（与开发共用，含种子数据）
- @Transactional 注解的测试方法自动回滚，避免污染 DB

7.3 单元测试覆盖

后端共 204 个测试用例，全数通过。按模块分布：

测试类	用例数	覆盖范围
AiChatServiceTest	12	AI-4 流路径 + ADMIN 拒绝 + LLM 失败降级
ApplicationServiceTest	19	投递 + 撤回 + 状态推进 + HR 备注 + AI-2 失败降级 + VIEWED_BY_HR 可见回归
ProfileServiceTest	20	个人资料 / 偏好 / 公司信息 + VERIFIED → PENDING 重置回归
JobServiceTest	36	职位 CRUD + AI-3 润色 + 收藏
ResumeServiceTest	10	上传 / 解析 / 删除 / 归档
AdminUserServiceTest	12	用户审核 + 角色管理
AdminCompanyServiceTest	8	公司审核 + 联动禁用
DictServiceTest	7	字典 CRUD
MessageServiceTest	8	发送 + 系统消息 + 附件绑定
ConversationServiceTest	11	二元组唯一 + 参与者鉴权 + 自动已读 + 推送
AttachmentServiceTest	6	上传 + 下载 + 鉴权 + bindToMessage
SseEmitterManagerTest	4	register / pushToUser / 自动 unregister
AuthServiceTest	6	注册 / 登录 / 验证码
LlmScoreServiceTest	5	AI-2 评分
LlmResumeParseServiceTest	8	AI-1 解析容错
LlmJdServiceTest	4	AI-3 润色
LlmChatServiceTest	13	AI-4 关键词检索 + topK + buildContext
LlmClientTest	3	流式 LLM 调用
LlmClientJsonExtractTest	11	JSON 抽取
ApplicationTests	1	Spring 上下文加载

7.4 关键测试用例

业务逻辑类覆盖 8 项核心场景：
(1) 投递状态机合法转换：PENDING_REVIEW → VIEWED_BY_HR → RESUME_PASSED → INTERVIEWING → OFFERED → HIRED 全链路推进成功；
(2) 投递状态机非法转换：PENDING_REVIEW → INTERVIEWING（跨级）抛 400；
(3) HR 跨职位推进：hrB 推进 hrA 职位的投递抛 403；
(4) 简历 AI 解析容错：缺失字段强制空数组、日期格式容错；
(5) 消息中心二元组唯一：同一对 HR ↔ 求职者多次调用 findOrCreate 返回同一会话；
(6) 消息中心自动已读：进入会话时把对方消息标 read_flag=1；
(7) 撤回系统消息失败不影响主流程：MessageService 抛异常时 withdraw 仍成功，application.status 已变 WITHDRAWN；
(8) 偏好清空真实落库：候选职位 / 行业 / 城市字段全部 null，DB 确实写入 NULL。

AI 集成类覆盖 6 项核心场景：
(1) AI-1 解析提示词强约束：模拟 LLM 响应缺字段 / 错误日期格式时 normalize 保障；
(2) AI-2 容错：LLM 返回 score 越界时 clamp 到 [0, 100]；
(3) AI-3 润色：5 关键词 → LLM 生成完整 JD 字段；
(4) AI-4 流式响应：5 阶段事件（ready/search_done/llm_start/delta/done）按序触发；
(5) AI-4 关键词检索阈值：未命中 → 固定提示词；(6) AI-4 prompt 敏感护栏——请求"如何 PENDING_REVIEW"应被拒绝。

7.5 测试结果

- **单元测试**：204 / 204 通过（100%）
- **集成测试**：@SpringBootTest 上下文加载成功，前后端运行成功，前后端无报错
- **mvn package**：构建成功

八、AI 工具使用情况说明

开发阶段	AI 工具	使用情况	成果形式
需求分析 / 系统设计	多模型辅助	协助梳理 8 大功能模块、辅助 ER 设计、与人工设计交叉验证	需求文档 + 8 份详细设计
后端开发	Minimax_M3	按模块逐个实现 service/ controller/mapper/test	13 个 service + 204 个测试
前端开发	Minimax_M3	实现 25 个 vue 页面 + 7 个 Pinia store + 12 个 API 模块	0 新警告 build
Bug 修复 / 重构	Minimax_M3	多轮迭代修复 AI 排序、标签门槛、撤回、列表刷新、状态可见性等	累计 6 段修复记录
文档与报告	多模型辅助	协助编写项目阶段报告、详细设计交叉校对	设计文档

九、总结

9.1 项目成果

本项目交付了一个端到端可演示的智能招聘 SaaS 原型。功能规模上，系统覆盖 8 大功能模块（账号体系、简历、职位、投递、消息、AI 集成、管理员、基础设施）、20 张数据表（覆盖 10 大主题领域，统一软删 + 关键索引）、14 个 REST 控制器 + 19 个业务服务（前后端分离 + JWT 鉴权 + Result 统一包装）、4 大 AI 功能（简历解析、简历↔JD 评分、JD 润色、智能客服，全部工程化调用 + 失败降级）、25 个前端页面（3 角色权限 + 完整业务流程 + SSE 实时推送）。质量保证方面，204 个后端单元测试全数通过。部署层面，系统提供完整部署链路：schema 初始化 + 增量迁移 + LLM Key 配置 + 本地一键启动。

9.2 技术亮点

- **AI 接口抽象层**：OpenAI 兼容协议，切换服务商仅改 3 个配置字段
- **4 维简历评分 + LLM 异步重排**：4 维（行业/城市/省份/关键词）Java 内存打分 + LLM 异步精排 + 内存 cache + SSE 增量推送
- **SSE 实时通道**：消息推送 + AI 评分结果推送；服务端心跳防 nginx 60s 超时
- **撤回事务隔离**：sendSystemMessage 用 REQUIRES_NEW 传播，消息失败不影响撤回主流程
- **8 状态投递机**：合法转换图严格校验；求职者 / HR 端 DTO 字段分离（求职者侧不暴露 AI 评分）
- **Spring Security crypto BCrypt**：密码哈希；自含盐
- **Vite 代理 + /api/* 转发**：前后端分离部署；SSE 端点 ?t=<token> 鉴权

9.3 不足与展望

类别	现状	后续
附件孤儿文件	DB 软删后磁盘文件不自动清理	增定时清理任务（按 N 天前创建 + 软删标记扫描）
AI 评分缓存	仅内存，进程重启后重算	接 Redis 持久化；或 DB 物化
简历详情候选	当前显示完整状态时间线（含 VIEWED_BY_HR）	部分用户偏好隐藏"处理中"状态——可加求职者偏好开关
AI-5 / AI-6 扩展项	原本设计的扩展项 AI 集成功能未实现	后续可增量开发，完善 AI 集成
全文搜索	职位搜索仅 LIKE，未上 ES	接 Elasticsearch 支持 JD / 简历全文检索
AI 偏好推荐	未增加由简历解析触发项	增加简历字段解析后自动推荐

参考文献

[1] Oracle Corporation. MySQL 8.0 reference manual[EB/OL]. [2026-10-08]. https://dev.mysql.com/doc/refman/8.0/en/.

[2] Vue.js. Vue.js guide[EB/OL]. [2026-10-08]. https://vuejs.org/guide/.

[5] Element Plus. Element Plus components documentation[EB/OL]. [2026-10-08]. https://element-plus.org/.

[4] Jones M. JSON Web Token (JWT): RFC 7519[S/OL]. IETF, 2015[2026-10-08]. https://datatracker.ietf.org/doc/html/rfc7519.

[5] Pinia. Pinia: the intuitive store for Vue.js[EB/OL]. [2026-10-08]. https://pinia.vuejs.org/.

[6] Vue.js. Composition API FAQ[EB/OL]. [2026-10-08]. https://vuejs.org/guide/extras/composition-api-faq.html.

[7] OpenAI. API reference[EB/OL]. [2026-10-08]. https://platform.openai.com/docs/api-reference.